

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



GRAPH DAN TREE

Oleh:

Muhammad Dani Maulana

NIM. 2510817110012

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph dan Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Dani Maulana
NIM 2510817110012

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	1
DAFTAR ISI.....	2
DAFTAR GAMBAR.....	3
DAFTAR TABEL.....	4
Soal 1: Konversi Adjacency List ke Adjacency Matrix.....	5
A. Source Code.....	7
B. Output Program	8
C. Pembahasan	8
D. Analisis	10
Soal 2: Konversi Adjacency Matrix ke Adjacency List	11
A. Source Code.....	12
B. Output Program	14
C. Pembahasan	14
D. Analisis	15
Soal 3: BFS: Jarak Terpendek Keluar Labirin	16
A. Source Code.....	18
B. Output Program	20
C. Pembahasan	20
D. Analisis	21
Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin	23
A. Source Code.....	25
B. Output Program	27
C. Pembahasan	27
D. Analisis	28
Soal 5: Tree Traversal	30
A. Source Code.....	31
B. Output Program	34
C. Pembahasan	34
D. Analisis	36

DAFTAR GAMBAR

Gambar 1.Output Soal_1.cpp.....	8
Gambar 2.Output Soal_2.cpp.....	14
Gambar 3.Output Soal_3.cpp.....	20
Gambar 4.Output Soal_4.cpp.....	27
Gambar 5.Output Soal_5.cpp.....	34

DAFTAR TABEL

Table 1. Source Code Soal_1.cpp.....	7
Table 2. Source Code Soal_2.cpp.....	12
Table 3. Source Code Soal_3.cpp.....	18
Table 4. Source Code Soal_4.cpp.....	25
Table 5. Source Code Soal_5.cpp.....	31

Soal 1: Konversi Adjacency List ke Adjacency Matrix

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U, V, W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U, V, W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi **adjacency matrix**. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
M
U1 V1 W1 U2
V2 W2
...
UM VM WM
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

```
Adjacency Matrix:
V1 V2 ... VN
V1 a11 a12 ... a1N V2
a21 a22 ... a2N
...
VN aN1 aN2 ... aNN
```

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki **self-loop**.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 6 A B 4 A C 2 B D 7 C B 1 C E 6 D C 3	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6 D 0 0 3 0 0 E 0 0 0 0 0

Penjelasan Contoh

Edge A B 4 menghasilkan nilai 4 pada baris A kolom B. Karena graph berarah, nilai pada baris B kolom A tetap 0 selama tidak ada edge dari B menuju A.

Edge D C 3 menghasilkan nilai 3 pada baris D kolom C. Vertex E tidak memiliki edge keluar, sehingga seluruh nilai pada baris E adalah 0.

A. Source Code

Table 1. Source Code Soal_1.cpp

```

1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      int N, M;
6
7      if (!(std::cin >> N)) return 0;
8      std::vector<char> labels(N);
9      for (int i = 0; i < N; ++i) {
10         std::cin >> labels[i];
11     }
12     std::vector<std::vector<int>> matrix(N, std::vector<int>(N,
0));
13
14     if (!(std::cin >> M)) return 0;
15     for (int i = 0; i < M; ++i) {
16         char u, v;
17         int w;
18         if (std::cin >> u >> v >> w) {
19             int u_idx = -1, v_idx = -1;
20             for (int k = 0; k < N; ++k) {
21                 if (labels[k] == u) u_idx = k;
22                 if (labels[k] == v) v_idx = k;
23             }
24             if (u_idx != -1 && v_idx != -1) {
25                 matrix[u_idx][v_idx] = w;
26             }
27         }
28     }
29
30     std::cout << "Adjacency Matrix:\n";
31     for (int i = 0; i < N; ++i) std::cout << " " << labels[i];

```



```

32     std::cout << std::endl;
33
34     for (int i = 0; i < N; ++i) {
35         std::cout << labels[i];
36         for (int j = 0; j < N; ++j) {
37             std::cout << " " << matrix[i][j];
38         }
39         std::cout << std::endl;
40     }
41     return 0;
42 }

```

B. Output Program



```

PS C:\Users\sabil\Documents\GitHub\module-6-graph-and-tree-berrypro08> &
wsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-yb3cjkdm.qxm' '--stdou
=Microsoft-MIEngine-Pid-ypo3njv4.5qa' '--dbgExe=C:\MinGW\bin\gdb.exe' '--i
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0

```

Gambar 1. Output Soal_1.cpp

C. Pembahasan

`#include <iostream>` dan `#include <vector>` digunakan untuk memanggil pustaka standar C++ agar program dapat melakukan operasi input-output dan menggunakan struktur data vektor yang fleksibel.

`int main()` { adalah titik awal eksekusi seluruh program.

`int N, M;` mendeklarasikan dua variabel integer untuk menyimpan jumlah vertex (N) dan jumlah sisi (M).

`if (!(std::cin >> N)) return 0;` membaca nilai N dari pengguna, jika input gagal, program berhenti seketika.

`std::vector<char> labels(N);` membuat vektor berukuran N untuk menyimpan karakter label setiap vertex.

`for (int i = 0; i < N; ++i) { std::cin >> labels[i]; }` adalah pengulangan untuk mengisi setiap posisi di vector label dengan karakter input dari pengguna.

`std::vector<std::vector<int>> matrix(N, std::vector<int>(N, 0));` membuat matriks dua dimensi berukuran N x N dan menginisialisasi seluruh isinya dengan angka 0.

`if (!(std::cin >> M)) return 0;` membaca jumlah sisi yang akan diproses, dengan pengecekan keamanan jika input kosong.

`for (int i = 0; i < M; ++i) {` memulai pengulangan sebanyak M kali untuk memproses setiap sisi graf.

`char u, v; int w;` mendeklarasikan variabel untuk menampung label vertex asal (u), vertex tujuan (v), dan bobot sisi (w).

`if (std::cin >> u >> v >> w) {` membaca ketiga nilai tersebut dari input pengguna.

`int u_idx = -1, v_idx = -1;` inisialisasi variabel indeks untuk mencari posisi label u dan v dalam matriks.

`for (int k = 0; k < N; ++k) {` mencari posisi indeks dari label yang diinput dengan cara membandingkannya satu per satu.

`if (labels[k] == u) u_idx = k;` jika label u ditemukan, simpan lokasinya di `u_idx`

`if (labels[k] == v) v_idx = k;` jika label v ditemukan, simpan lokasinya di `v_idx`.

`if (u_idx != -1 && v_idx != -1) { matrix[u_idx][v_idx] = w; }` jika kedua label valid ditemukan, masukkan bobot w ke dalam sel matriks yang sesuai.

`std::cout << "Adjacency Matrix:\n";` mencetak judul ke layar sebagai informasi hasil.

`for (int i = 0; i < N; ++i) std::cout << " " << labels[i]; std::cout << std::endl;` mencetak baris label kolom agar matriks lebih mudah dibaca manusia.

`for (int i = 0; i < N; ++i) {` memulai pengulangan untuk mencetak isi matriks baris demi baris.

`std::cout << labels[i];` mencetak label vertex di awal setiap baris matriks.

`for (int j = 0; j < N; ++j) { std::cout << " " << matrix[i][j]; }` mencetak nilai bobot yang ada di dalam setiap kolom matriks.

`std::cout << std::endl; }` mencetak baris baru setelah setiap baris matriks selesai dicetak.

`return 0; }` mengakhiri fungsi main dan menandakan program selesai.

D. Analisis

Algoritma ini membangun *adjacency matrix* melalui tiga tahap utama: pemetaan, translasi, dan konstruksi. Pertama, program menyimpan label vertex ke dalam vektor sebagai referensi indeks. Kedua, saat membaca sisi, program melakukan pencarian linear untuk menerjemahkan label ke koordinat matriks (baris, kolom). Ketiga, bobot sisi ditempatkan pada koordinat tersebut dalam matriks $N \times N$ yang telah diinisialisasi dengan nol. Hasil akhirnya adalah tabel numerik yang merepresentasikan struktur graf secara terarah. Kompleksitas waktu dari algoritma ini adalah $O(M \times N)$, di mana M adalah jumlah sisi dan N adalah jumlah vertex. Hal ini disebabkan oleh proses pencarian label, untuk setiap M sisi yang dibaca, program melakukan perulangan for hingga N kali guna menemukan posisi indeks dari dua vertex yang terlibat.

adanya deklarasi matriks `matrix(N, std::vector<int>(N, 0))` menyebabkan matriks ini menjadi variabel dominan yang menentukan batas atas penggunaan ruang memori. Sehingga penggunaan memori sekaligus kompleksitas ruangnya sebesar $O(N^2)$.

Soal 2: Konversi Adjacency Matrix ke Adjacency List

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang direpresentasikan dalam bentuk **adjacency matrix** berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi **adjacency list**. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
A11 A12 ... A1N
A21 A22 ... A2N
...
AN1 AN2 ... ANN
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

```
Adjacency List:
V1 -> (X1,w1), (X2,w2), ...
V2 -> (X1,w1), (X2,w2), ...
...
VN -> ...
```

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
<pre> 5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0 </pre>	<pre> Adjacency List: A -> (B,4), (C,2) B -> (D,7) C -> (B,1), (E,6) D -> (C,3) E -> - </pre>

Penjelasan Contoh

Pada baris vertex A, nilai positif muncul pada kolom B dan C. Karena itu, adjacency list untuk A adalah (B,4), (C,2).

Pada baris vertex E, semua nilai bernilai 0, sehingga vertex E tidak memiliki edge keluar dan output-nya adalah E -> -.

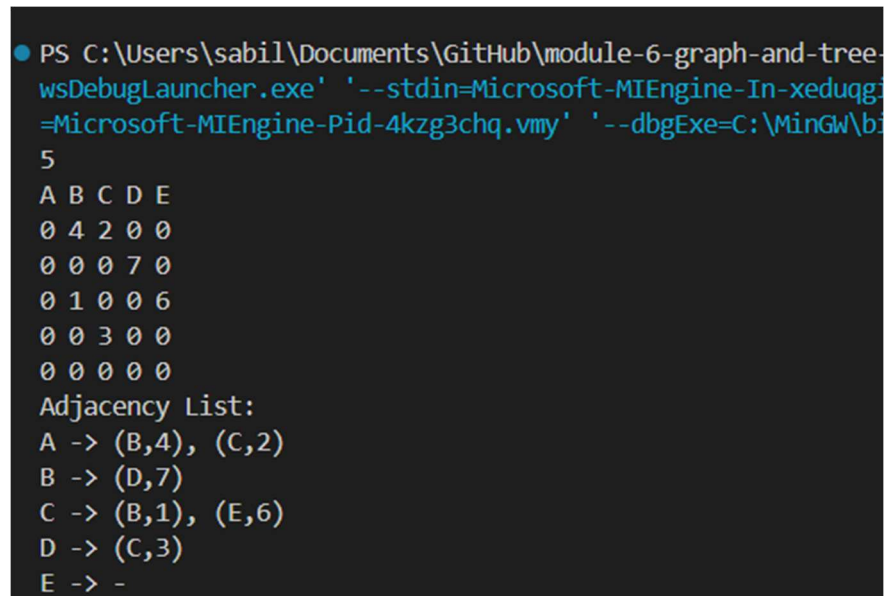
A. Source Code

Table 2. Source Code Soal_2.cpp

1	#include <iostream>
2	#include <vector>
3	
4	int main() {
5	int N;
6	if (!(std::cin >> N) N <= 0) return 0;
7	
8	std::vector<char> labels(N);
9	for (int i = 0; i < N; ++i) {
10	std::cin >> labels[i];
11	}
12	

```
13     std::vector<std::vector<int>> matrix(N,  
std::vector<int>(N));  
14     for (int i = 0; i < N; ++i) {  
15         for (int j = 0; j < N; ++j) {  
16             std::cin >> matrix[i][j];  
17         }  
18     }  
19  
20     std::cout << "Adjacency List:\n";  
21     for (int i = 0; i < N; ++i) {  
22         std::cout << labels[i] << " -> ";  
23         bool first = true;  
24         bool Edge = false;  
25         for (int j = 0; j < N; ++j) {  
26             if (matrix[i][j] != 0) {  
27                 if (!first) std::cout << ", ";  
28                 std::cout << "(" << labels[j] << "," <<  
matrix[i][j] << ")";  
29                 first = false;  
30                 Edge = true;  
31             }  
32         }  
33         if (!Edge) {  
34             std::cout << "-";  
35         }  
36         std::cout << std::endl;  
37     }  
38  
39     return 0;  
40 }
```

B. Output Program



```

PS C:\Users\sabil\Documents\GitHub\module-6-graph-and-tree-  
wsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-xeduqg  
=Microsoft-MIEngine-Pid-4kzg3chq.vmy' '--dbgExe=C:\MinGW\bi  
5  
A B C D E  
0 4 2 0 0  
0 0 0 7 0  
0 1 0 0 6  
0 0 3 0 0  
0 0 0 0 0  
Adjacency List:  
A -> (B,4), (C,2)  
B -> (D,7)  
C -> (B,1), (E,6)  
D -> (C,3)  
E -> -

```

Gambar 2. Output Soal_2.cpp

C. Pembahasan

`#include <iostream>` dan `<vector>` memuat pustaka untuk input-output dan penggunaan kontainer `std::vector`.

`int main()` { adalah fungsi utama tempat program mulai dieksekusi.

`int N;` mendeklarasikan variabel untuk menampung jumlah vertex.

`if (!(std::cin >> N) || N <= 0) return 0;` membaca input jumlah vertex dan menghentikan program jika input tidak valid atau N kurang dari 1.

`std::vector<char> labels(N);` membuat vektor untuk menyimpan label karakter setiap vertex.

`for (int i = 0; i < N; ++i) { std::cin >> labels[i]; }` mengisi vektor labels dengan karakter vertex berdasarkan input pengguna.

`std::vector<std::vector<int>> matrix(N, std::vector<int>(N));` menyiapkan matriks dua dimensi berukuran N x N untuk menyimpan data.

`for (int i = 0; i < N; ++i) { for (int j = 0; j < N; ++j) { std::cin >> matrix[i][j]; } }` membaca seluruh isi matriks dari input pengguna, baris demi baris.

`std::cout << "Adjacency List:\n";` mencetak judul output ke layar.

`for (int i = 0; i < N; ++i) {` memulai perulangan untuk memproses setiap vertex asal (i).

```

std::cout << labels[i] << " -> "; mencetak label vertex asal sebagai titik awal daftar.
bool first = true; dan bool Edge = false; menginisialisasi penanda untuk
mengatur format tampilan koma dan mendeteksi apakah ada sisi yang ditemukan.
for (int j = 0; j < N; ++j) { memulai perulangan kolom untuk mengecek hubungan
dari vertex i ke vertex j.
if (matrix[i][j] != 0) { memeriksa apakah terdapat sisi (nilai bukan nol) pada
koordinat tersebut.
if (!first) std::cout << ", "; menambahkan koma sebagai pemisah jika ini bukan
hubungan pertama yang dicetak.
std::cout << "(" << labels[j] << ", " << matrix[i][j] << ")"; mencetak label
tujuan dan bobot sisi dalam format pasangan.
first = false; dan Edge = true; memperbarui status bahwa data pertama sudah
dicetak dan setidaknya satu sisi ditemukan.
if (!Edge) { std::cout << "-"; } jika tidak ditemukan sisi sama sekali (Edge tetap
false), maka cetak tanda -.
std::cout << std::endl; } mencetak baris baru untuk pindah ke vertex berikutnya.
return 0; } menutup fungsi main dan mengakhiri program.

```

D. Analisis

Algoritma ini bekerja dengan melakukan konversi data graf dari format matriks ke format Adjacency List. Proses utamanya melibatkan iterasi ganda untuk menelusuri setiap sel dalam matriks $N \times N$. Pada setiap sel, algoritma memeriksa apakah nilai bobot bukan nol jika ditemukan, artinya terdapat sisi yang menghubungkan vertex baris ke vertex kolom. Program kemudian mencetak detail hubungan tersebut menggunakan label vertex yang sesuai dan menandai keberadaan sisi untuk mengatur format tampilan (seperti penambahan koma antar hubungan atau tanda hubung jika tidak ada koneksi).

program wajib memeriksa setiap elemen dalam matriks $N \times N$ untuk menentukan apakah terdapat koneksi, terlepas dari seberapa sparse atau dense graf tersebut. Jika graf memiliki N vertex, maka terdapat N^2 sel yang harus diproses oleh perulangan ganda. Maka dari itu, Kompleksitas waktu dari algoritma ini adalah $O(N^2)$.

adanya alokasi matriks $N \times N$ di awal program menyebabkan penggunaan memori sebesar $O(N^2)$, sehingga kompleksitas ruangnya adalah $O(N^2)$.

Soal 3: BFS: Jarak Terpendek Keluar Labirin

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma **Breadth-First Search** (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21 G22 ... G2C
...
GR1 GR2 ... GRC
SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

```
X
```

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
<pre> 8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9 </pre>	<pre> 16 </pre>

Penjelasan Contoh

Posisi awal berada pada $(0, 0)$ dan posisi tujuan berada pada $(7, 9)$. Labirin memiliki beberapa percabangan dan jalan buntu, misalnya cabang menuju area kiri bawah dan cabang menuju sel $(2, 1)$.

Dengan BFS, setiap sel diproses berdasarkan jarak terdekat dari start. Jarak minimum dari $(0, 0)$ ke $(7, 9)$ adalah 16 langkah, sehingga output yang dicetak hanya 16.

A. Source Code

Table 3. Source Code Soal_3.cpp

```

1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5  struct Node {
6      int r, c, dist;
7  };
8
9  int main() {
10     int R, C;
11     if (!(cin >> R >> C) || R <= 0 || C <= 0) return 0;
12
13     vector<vector<int>> grid(R, vector<int>(C));
14     for (int i = 0; i < R; ++i) {
15         for (int j = 0; j < C; ++j) {
16             cin >> grid[i][j];
17         }
18     }
19
20     int SR, SC, FR, FC;
21     if (!(cin >> SR >> SC >> FR >> FC)) return 0;
22     if (SR < 0 || SR >= R || SC < 0 || SC >= C || FR < 0 || FR
23     >= R || FC < 0 || FC >= C) {
24         return 0;
25     }
26
27     vector<Node> q;
28     int head = 0;
29
30     vector<vector<bool>> visited(R, vector<bool>(C, false));
31
32     q.push_back({SR, SC, 0});

```

```
32     visited[SR][SC] = true;
33
34     int dr[] = {-1, 1, 0, 0};
35     int dc[] = {0, 0, -1, 1};
36
37     while (head < q.size()) {
38         Node curr = q[head];
39         head++;
40
41         if (curr.r == FR && curr.c == FC) {
42             cout << curr.dist << endl;
43             return 0;
44         }
45
46         for (int i = 0; i < 4; ++i) {
47             int nr = curr.r + dr[i];
48             int nc = curr.c + dc[i];
49
50             if (nr >= 0 && nr < R && nc >= 0 && nc < C &&
grid[nr][nc] == 0 && !visited[nr][nc]) {
51                 visited[nr][nc] = true;
52                 q.push_back({nr, nc, curr.dist + 1});
53             }
54         }
55     }
56     cout << -1 << endl;
57     return 0;
58 }
```

B. Output Program

```
PS C:\Users\sabil\Documents\GitHub\module-6-graph-and-tree-berrypro08> & 'c:\Users\sabil\Documents\GitHub\module-6-graph-and-tree-berrypro08\wsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-hozotsqm.aze' '--stdout=Microsoft-MIEngine-Pid-22ylzltz.lkb' '--dbgExe=C:\MinGW\bin\gdb.exe' '--interpreter=
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16
```

Gambar 3. Output Soal_3.cpp

C. Pembahasan

`#include <iostream>` dan `<vector>` memuat pustaka untuk input/output dan penggunaan vektor.

`using namespace std;` memudahkan penulisan perintah standar seperti `cin` dan `cout`.

`struct Node { int r, c, dist; };` mendefinisikan struktur data untuk menyimpan koordinat baris (`r`), kolom (`c`), dan jarak tempuh (`dist`).

`int R, C;` mendeklarasikan variabel untuk ukuran grid.

`if (!(cin >> R >> C) || R <= 0 || C <= 0) return 0;` membaca ukuran grid dan menghentikan program jika input tidak valid.

`vector<vector<int>> grid(R, vector<int>(C));` membuat matriks dua dimensi untuk peta grid.

`for` ganda membaca nilai setiap sel grid dari input pengguna.

`int SR, SC, FR, FC;` mendeklarasikan titik awal (`S`) dan titik tujuan (`F`).

`if (!(cin >> SR >> SC >> FR >> FC)) return 0;` membaca koordinat awal dan akhir.

`if (SR < 0 || ... ) return 0;` melakukan validasi agar koordinat tidak di luar jangkauan grid.

`vector<Node> q;` membuat vektor untuk difungsikan sebagai antrean BFS.

`int head = 0;` penanda posisi awal antrean yang sedang diproses.

```
vector<vector<bool>> visited(R, vector<bool>(C, false)); matriks untuk
menandai sel yang sudah dikunjungi agar tidak diproses berulang.
q.push_back({SR, SC, 0}); memasukkan titik awal ke antrean dengan jarak awal 0.
visited[SR][SC] = true; menandai titik awal sebagai sel yang sudah dikunjungi.
int dr[] = {-1, 1, 0, 0}; dan int dc[] = {0, 0, -1, 1}; array pendukung untuk
pergerakan empat arah (atas, bawah, kiri, kanan).
while (head < q.size()) { memulai perulangan utama selama antrean belum kosong.
Node curr = q[head]; head++; mengambil elemen dari antrean dan menggeser penanda
head.
if (curr.r == FR && curr.c == FC) { cout << curr.dist << endl; return 0; }
mengecek apakah titik saat ini adalah tujuan; jika ya, cetak jarak dan selesai.
for (int i = 0; i < 4; ++i) { memulai perulangan untuk mengecek empat adjacent
vertices sel saat ini.
int nr = curr.r + dr[i]; int nc = curr.c + dc[i]; menghitung koordinat adjacent
vertices baru.
if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc] == 0 &&
!visited[nr][nc]) { memastikan adjacent vertices berada di dalam batas adalah jalur
kosong dan belum dikunjungi.
visited[nr][nc] = true; q.push_back({nr, nc, curr.dist + 1}); menandai
adjacent vertices sebagai sudah dikunjungi dan menambahkannya ke antrean dengan jarak
bertambah 1.
cout << -1 << endl; mencetak -1 jika perulangan selesai tanpa menemukan titik tujuan.
return 0; } mengakhiri fungsi main.
```

D. Analisis

Algoritma ini menggunakan metode *Breadth-First Search* (BFS) untuk mencari jalur terpendek dalam grid dua dimensi, dengan bekerja secara sistematis menyebar dari titik awal ke arah adjacent vertices terdekat secara berlapis. Proses diawali dengan memasukkan titik awal ke dalam antrean, lalu secara berulang mengambil vertex dari antrean tersebut untuk diperiksa adjacent verticesnya ke empat arah mata angin. Algoritma memanfaatkan matriks *visited* untuk mencatat sel yang sudah pernah dilewati, sehingga mencegah pengulangan

proses pada sel yang sama dan menghindari *looping* yang tidak perlu. Karena sifat BFS yang menjelajahi setiap sel berdasarkan jarak langkah dari titik awal, vertex tujuan yang ditemukan pertama kali dipastikan merupakan jalur terpendek yang mungkin, dengan total kompleksitas waktu sebesar $O(R \times C)$ karena setiap sel pada grid akan dikunjungi maksimal satu kali.

semua struktur data utama yang digunakan skalanya berbanding lurus dengan jumlah sel dalam *grid* ($N = R \times C$), maka total kompleksitas ruangnya adalah $O(R \times C)$.

Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin

Time Limit: 2.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma **Depth-First Search** (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses **backtracking** agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21 G22 ... G2C
...
GR1 GR2 ... GRC
SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

```
T
```

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	4

Penjelasan Contoh

Posisi awal berada pada $(0,0)$ dan posisi tujuan berada pada $(4,5)$. Labirin memiliki lebih dari satu jalur valid serta beberapa cabang yang tidak menghasilkan solusi, misalnya cabang menuju sel $(0,5)$.

Dengan DFS dan backtracking, seluruh jalur sederhana dari start ke finish dapat dihitung. Pada contoh ini terdapat 4 jalur valid, sehingga output yang dicetak hanya 4.

A. Source Code

Table 4. Source Code Soal_4.cpp

```

1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5  struct DataPosisi {
6      int r, c;
7      int arahKe;
8  };
9
10 int main() {
11     int R, C;
12     cin >> R >> C;
13     vector<vector<int>> grid(R, vector<int>(C));
14     for (int i = 0; i < R; ++i) {
15         for (int j = 0; j < C; ++j) {
16             cin >> grid[i][j];
17         }
18     }
19     int SR, SC, FR, FC;
20     cin >> SR >> SC >> FR >> FC;
21     vector<vector<bool>> visited(R, vector<bool>(C, false));
22     vector<DataPosisi> tumpukan;
23     int jumlahJalur = 0;
24
25     int dr[] = {-1, 1, 0, 0};
26     int dc[] = {0, 0, -1, 1};
27
28     tumpukan.push_back({SR, SC, 0});
29     visited[SR][SC] = true;
30
31     while (!tumpukan.empty()) {
32         int r = tumpukan.back().r;

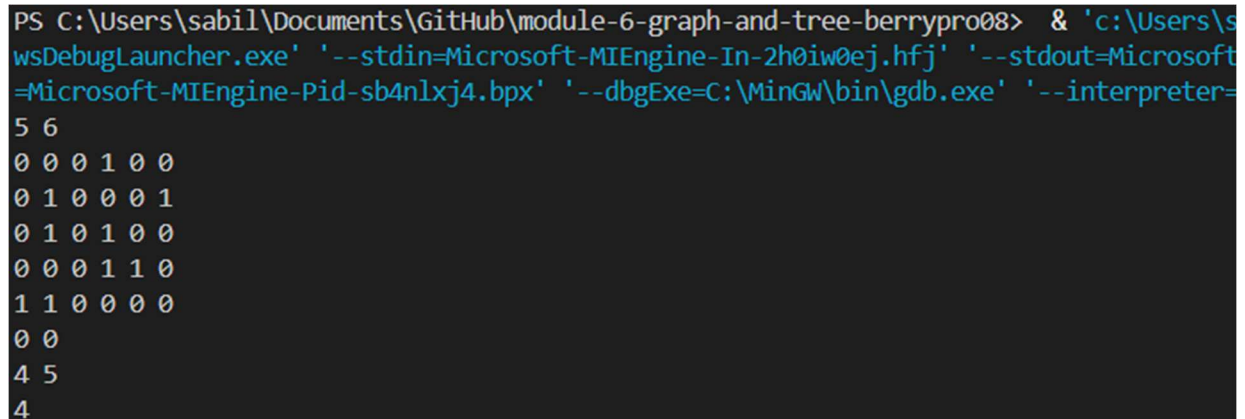
```

```

33         int c = tumpukan.back().c;
34         int d = tumpukan.back().arahKe;
35
36         if (r == FR && c == FC) {
37             jumlahJalur++;
38             visited[r][c] = false;
39             tumpukan.pop_back();
40             continue;
41         }
42         if (tumpukan.back().arahKe < 4) {
43             int arahSekarang = tumpukan.back().arahKe;
44             tumpukan.back().arahKe++;
45
46             int barisBaru = r + dr[arahSekarang];
47             int kolomBaru = c + dc[arahSekarang];
48             if (barisBaru >= 0 && barisBaru < R && kolomBaru >=
0 && kolomBaru < C &&
49                 grid[barisBaru][kolomBaru] == 0 &&
!visited[barisBaru][kolomBaru]) {
50
51                 visited[barisBaru][kolomBaru] = true;
52                 tumpukan.push_back({barisBaru, kolomBaru, 0});
53             }
54         } else {
55             visited[r][c] = false;
56             tumpukan.pop_back();
57         }
58     }
59     cout << jumlahJalur << endl;
60
61     return 0;
62 }

```

B. Output Program



```
PS C:\Users\sabil\Documents\GitHub\module-6-graph-and-tree-berrypro08> & 'c:\Users\sabil\Documents\GitHub\module-6-graph-and-tree-berrypro08\Debug\wsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-2h0iw0ej.hfj' '--stdout=Microsoft-MIEngine-Pid-sb4nlxj4.bpx' '--dbgExe=C:\MinGW\bin\gdb.exe' '--interpreter=
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
```

Gambar 4. Output Soal_4.cpp

C. Pembahasan

`#include <iostream>` dan `<vector>` memuat pustaka untuk input-output dan penggunaan struktur data vektor.

`using namespace std;` memudahkan penulisan perintah standar seperti `cin` dan `cout`.

`struct DataPosisi { int r, c, arahKe; };` mendefinisikan struktur untuk menyimpan koordinat baris (`r`), kolom (`c`), dan indeks arah berikutnya yang akan dieksplorasi (`arahKe`).

`int R, C; cin >> R >> C;` mendeklarasikan dan membaca dimensi grid dari pengguna.

`vector<vector<int>> grid(R, vector<int>(C));` mengalokasikan matriks dua dimensi untuk memetakan jalur (0) dan hambatan.

`for` ganda membaca nilai isi grid dari input.

`int SR, SC, FR, FC; cin >> SR >> SC >> FR >> FC;` membaca koordinat titik awal dan titik tujuan.

`vector<vector<bool>> visited(R, vector<bool>(C, false));` matriks untuk melacak sel yang sedang dalam jalur pencarian aktif.

`vector<DataPosisi> tumpukan; int jumlahJalur = 0;` menyiapkan tumpukan untuk DFS dan penghitung jumlah jalur.

`int dr[] = {-1, 1, 0, 0}; int dc[] = {0, 0, -1, 1};` array *offset* untuk pergerakan empat arah (atas, bawah, kiri, kanan).

`tumpukan.push_back({SR, SC, 0}); visited[SR][SC] = true;` memasukkan posisi awal ke tumpukan dan menandainya sebagai sudah dikunjungi.

while (!tumpukan.empty()) { memulai perulangan utama selama tumpukan eksplorasi tidak kosong.

int r = tumpukan.back().r; int c = tumpukan.back().c; mengambil posisi koordinat dari elemen paling atas tumpukan.

if (r == FR && c == FC) { memeriksa apakah posisi saat ini adalah titik tujuan.

jumlahJalur++; visited[r][c] = false; tumpukan.pop_back(); continue; } jika sampai di tujuan, tambahkan hitungan, hapus dari tumpukan (untuk mencari jalur lain), dan lanjut ke iterasi berikutnya.

if (tumpukan.back().arahKe < 4) { memeriksa apakah masih ada arah yang belum dicoba dari sel tersebut.

int arahSekarang = tumpukan.back().arahKe; tumpukan.back().arahKe++; menyimpan arah saat ini dan menaikkan indeks arah untuk langkah selanjutnya.

int barisBaru = r + dr[arahSekarang]; int kolomBaru = c + dc[arahSekarang]; menghitung koordinat sel baru berdasarkan arah yang dipilih.

if (barisBaru >= 0 && barisBaru < R && kolomBaru >= 0 && kolomBaru < C && grid[barisBaru][kolomBaru] == 0 && !visited[barisBaru][kolomBaru]) { memvalidasi sel baru (dalam batas, bisa dilalui, belum dikunjungi).

visited[barisBaru][kolomBaru] = true; tumpukan.push_back({barisBaru, kolomBaru, 0}); jika valid, tandai sebagai dikunjungi dan masukkan ke tumpukan.

} else { visited[r][c] = false; tumpukan.pop_back(); } jika tidak ada arah valid lagi, lakukan *backtracking* (hapus dari tumpukan dan buka akses sel tersebut untuk jalur lain).

cout << jumlahJalur << endl; mencetak total jalur yang ditemukan.

return 0; } mengakhiri fungsi main.

D. Analisis

algoritma yang digunakan adalah **DFS dengan *backtracking*** yang berfungsi menghitung seluruh jumlah rute dari titik awal ke tujuan. Cara kerjanya adalah dengan menjelajahi setiap rute sedalam mungkin, saat mencapai tujuan, algoritma menambah jumlah jalur, lalu melakukan *backtracking* yaitu membatalkan status kunjungan pada sel tersebut agar rute alternatif lain dapat menggunakan sel yang sama. Karena pendekatan ini mencoba semua kombinasi langkah tanpa membatasi kunjungan sel, kompleksitas waktunya menjadi

eksponensial, yakni $O(4^{R \times C})$, di mana setiap sel di dalam grid memiliki hingga empat potensi arah percabangan yang harus dievaluasi. Akibatnya, durasi eksekusi akan meningkat drastis seiring bertambahnya ukuran grid.

Secara keseluruhan, karena semua struktur data yang digunakan matriks *grid*, matriks *visited*, dan *stack* skalanya berbanding lurus dengan total jumlah sel ($R \times C$), maka kompleksitas ruang totalnya adalah $O(R \times C)$.

Soal 5: Tree Traversal

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah **Red-Black Tree** secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- `PREORDER`, yaitu urutan root, subtree kiri, lalu subtree kanan.
- `INORDER`, yaitu urutan subtree kiri, root, lalu subtree kanan.
- `POSTORDER`, yaitu urutan subtree kiri, subtree kanan, lalu root.
- `ALL`, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

```
N
A1 A2 ... AN Q
T1 T2
... TQ
```

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

```
[Preorder]    :  daftar_angka
[Inorder]     :  daftar_angka
[Postorder]   :  daftar_angka
```

Jika query adalah `ALL`, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

```
Tree kosong. Tidak ada yang bisa ditampilkan.
```

Constraints

- $0 \leq N \leq 1000$
- $-10000000000 \leq A_i \leq 10000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder] : 20 40 30 60 80 70 50
PREORDER	[Preorder] : 50 30 20 40 70 60 80
INORDER	[Inorder] : 20 30 40 50 60 70 80
POSTORDER	[Postorder] : 20 40 30 60 80 70 50
ALL	

Penjelasan Contoh

Bilangan dimasukkan ke RBTree secara berurutan. Nilai 50 menjadi root. Nilai yang lebih kecil dari 50 masuk ke subtree kiri, sedangkan nilai yang lebih besar masuk ke subtree kanan.

Hasil INORDER pada RBTree selalu menghasilkan urutan nilai dari kecil ke besar. Pada contoh ini, hasil inorder adalah 20 30 40 50 60 70 80, sehingga struktur RBTree yang dibentuk sudah konsisten dengan aturan RBTree.

A. Source Code

Table 5. Source Code Soal_5.cpp

1	#include <iostream>
2	#include <string>
3	#include "RedBlackTree.h"
4	
5	using namespace std;
6	
7	void cetakPreorder(const RedBlackTree::Node* node, bool &first) {
8	if (node->isNil) return;
9	
10	if (!first) cout << " ";
11	cout << node->key;
12	first = false;
13	
14	cetakPreorder(node->left, first);


```
15     cetakPreorder(node->right, first);
16 }
17
18 void cetakInorder(const RedBlackTree::Node* node, bool &first) {
19     if (node->isNil) return;
20
21     cetakInorder(node->left, first);
22
23     if (!first) cout << " ";
24     cout << node->key;
25     first = false;
26
27     cetakInorder(node->right, first);
28 }
29
30 void cetakPostorder(const RedBlackTree::Node* node, bool &first) {
31     if (node->isNil) return;
32
33     cetakPostorder(node->left, first);
34     cetakPostorder(node->right, first);
35
36     if (!first) cout << " ";
37     cout << node->key;
38     first = false;
39 }
40
41 int main() {
42     RedBlackTree rbt;
43     int N;
44     if (!(cin >> N)) return 0;
45     for (int i = 0; i < N; i++) {
46         int angka;
47         cin >> angka;
48         if (!rbt.contains(angka)) {
```

```

49         rbt.insert(angka);
50     }
51 }
52
53 int Q;
54 cin >> Q;
55 for (int i = 0; i < Q; i++) {
56     string tipeQuery;
57     cin >> tipeQuery;
58
59     bool first = true;
60     if (tipeQuery == "PREORDER") {
61         cout << "[Preorder]   : ";
62         cetakPreorder(rbt.root(), first);
63         cout << endl;
64     } else if (tipeQuery == "INORDER") {
65         cout << "[Inorder]     : ";
66         cetakInorder(rbt.root(), first);
67         cout << endl;
68     } else if (tipeQuery == "POSTORDER") {
69         cout << "[Postorder]  : ";
70         cetakPostorder(rbt.root(), first);
71         cout << endl;
72     } else if (tipeQuery == "ALL") {
73         first = true;
74         cout << "[Preorder]   : ";
75         cetakPreorder(rbt.root(), first);
76         cout << endl;
77         first = true;
78         cout << "[Inorder]     : ";
79         cetakInorder(rbt.root(), first);
80         cout << endl;
81         first = true;
82         cout << "[Postorder]  : ";

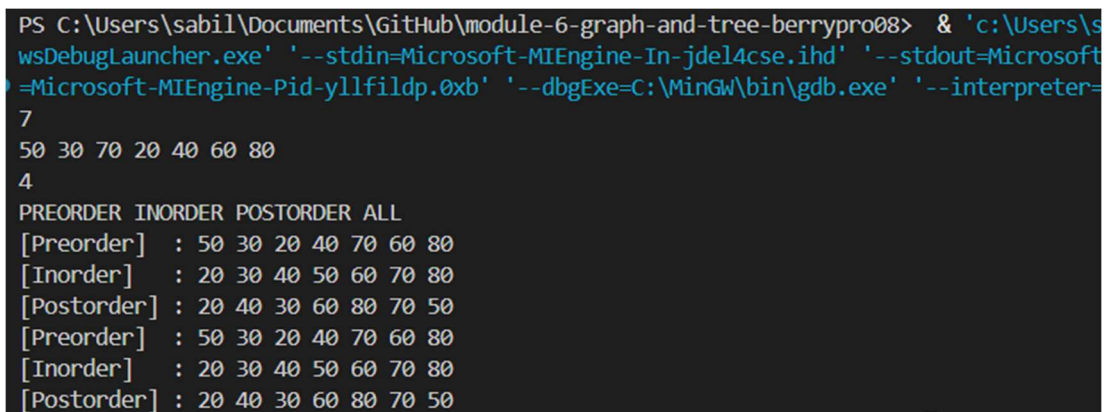
```

```

83         cetakPostorder(rbt.root(), first);
84         cout << endl;
85     }
86 }
87
88     return 0;
89 }

```

B. Output Program



```

PS C:\Users\sabil\Documents\GitHub\module-6-graph-and-tree-berrypro08> & 'c:\Users\sabil\Documents\GitHub\module-6-graph-and-tree-berrypro08\Debug\wsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-jde14cse.ihd' '--stdout=Microsoft-MIEngine-Pid-yllfildp.0xb' '--dbgExe=C:\MinGW\bin\gdb.exe' '--interpreter=
7
50 30 70 20 40 60 80
4
PREORDER INORDER POSTORDER ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50

```

Gambar 5. Output Soal_5.cpp

C. Pembahasan

#include <iostream>, <string>, "RedBlackTree.h": Memuat pustaka input-output, string, dan definisi struktur data RBT.

```
void cetakPreorder(const RedBlackTree::Node* node, bool &first) {:
```

Mendefinisikan fungsi rekursif Preorder (Root-Left-Right).

```
if (node->isNil) return;: Base case; berhenti jika simpul adalah NIL (kosong/daun).
```

```
if (!first) cout << " ";: Mencetak spasi pemisah jika bukan angka pertama.
```

```
cout << node->key; first = false;: Mencetak nilai kunci dan menandai bahwa angka pertama telah diproses.
```

```
cetakPreorder(node->left, first); cetakPreorder(node->right, first);:
```

Pemanggilan rekursif ke anak kiri lalu ke kanan.

```
void cetakInorder(...): Sama dengan Preorder, namun cout dilakukan di antara pemanggilan rekursif kiri dan kanan (Left-Root-Right).
```

`void cetakPostorder(...):` Sama dengan Preorder, namun `cout` dilakukan setelah kedua pemanggilan rekursif (Left-Right-Root).

`int main() {:` Mendefinisikan titik awal eksekusi program.

`RedBlackTree rbt;:` Membuat objek bernama `rbt` dari kelas `RedBlackTree` untuk menyimpan data dalam struktur pohon.

`int N;:` Mendeklarasikan variabel `N` untuk menyimpan jumlah data yang akan dimasukkan ke pohon.

`if (!(cin >> N)) return 0;:` Membaca jumlah data dari input pengguna; jika input gagal atau kosong, program langsung berhenti.

`for (int i = 0; i < N; i++) {:` Memulai perulangan untuk membaca `N` buah angka dari *standard input*.

`int angka; cin >> angka;:` Membaca satu per satu angka yang diinputkan pengguna.

`if (!rbt.contains(angka)) { rbt.insert(angka); }` Fungsi krusial untuk menjaga integritas data; angka hanya akan disisipkan ke dalam pohon jika belum ada di dalamnya (mencegah duplikasi kunci).

`int Q; cin >> Q;:` Membaca jumlah total perintah (*query*) yang akan dieksekusi setelah pohon terbentuk.

`for (int i = 0; i < Q; i++) {:` Memulai perulangan untuk memproses setiap perintah satu per satu.

`string tipeQuery; cin >> tipeQuery;:` Membaca perintah dalam bentuk teks (misalnya: "PREORDER", "INORDER", "POSTORDER", atau "ALL").

`bool first = true;:` Menginisialisasi variabel penanda spasi dengan `true`. Ini akan memastikan bahwa saat mencetak angka pertama, tidak ada spasi di depannya.

`if (tipeQuery == "PREORDER") { ... }` Mengecek perintah; jika "PREORDER", cetak label dan panggil fungsi `cetakPreorder` menggunakan *root* pohon.

`else if (tipeQuery == "INORDER") { ... }` Jika perintah "INORDER", panggil fungsi `cetakInorder`.

`else if (tipeQuery == "POSTORDER") { ... }` Jika perintah "POSTORDER", panggil fungsi `cetakPostorder`.

`else if (tipeQuery == "ALL") { ... }` Jika perintah "ALL", jalankan ketiga fungsi penelusuran (Preorder, Inorder, dan Postorder) secara berurutan. Di sini, `first = true`

dipanggil ulang sebelum setiap fungsi agar pemformatan spasi di setiap baris output selalu bersih.

`return 0;` Mengakhiri fungsi main dan mengembalikan nilai 0 ke sistem, yang menandakan bahwa program telah berjalan dengan sukses tanpa *error*.

D. Analisis

Mekanisme yang dipakai adalah *tree traversal* pada *Red-Black Tree* dengan menggunakan rekursi untuk menelusuri setiap node, di mana setiap metode (*Preorder*, *Inorder*, dan *Postorder*) memiliki urutan kunjungan yang berbeda berdasarkan prioritas kunjungan antara akar, cabang kiri, dan cabang kanan. Kompleksitas waktunya adalah $O(N)$ karena algoritma wajib mengunjungi seluruh node tree sebanyak satu kali untuk keperluan print di layar, sementara kompleksitas ruangnya adalah $O(\log N)$ karena sifat *Red-Black Tree* yang selalu menjaga keseimbangan tinggi pohon melalui rotasi otomatis. Pada fungsi main, program mengelola alur dengan membaca N data yang disaring melalui metode `contains`, kemudian merespons Q perintah *query* secara dinamis, di mana variabel `bool &first` diterapkan sebagai pengendali pemformatan spasi agar setiap *output* tetap rapi dan tidak menyertakan spasi di awal atau akhir baris.